

Bridging Semantics and Physics with Constrained LLMs for Safe and Trustworthy Robotic Manipulation

Wenhao Hong¹, Lan Wei¹, and Dandan Zhang^{1*}

Imperial College London, London, UK
d.zhang17@imperial.ac.uk

Abstract. A language-guided robot operating in a real kitchen must do more than produce a plan that appears correct. It must also execute that plan safely in cluttered environments under imperfect perception. Large language models (LLM) can decompose instructions into action sequences, yet a language-action gap remains: a plan may appear valid linguistically while being physically infeasible under kinematic and collision constraints. We bridge this gap by formalizing the reasoning-execution boundary as a typed contract. From RGB-D observations, the system grounds perceived objects in an explicit, collision-aware scene model and constrains language-level decisions through schema-validated tool calls defined by the Model Context Protocol (MCP), rejecting malformed commands before they reach the robot. Each validated call is deterministically grounded in a MoveIt Task Constructor pipeline, where candidate motions are evaluated against the reconstructed planning scene in a verify-then-act step. Only trajectories that pass both kinematic and collision checks are sent to the robot. On a physical UFactory 850, the method achieves up to 80% success across ten trials per task on pouring tasks involving liquids, granular media, and discrete solids. It achieves 90% success on a grasp-and-place task using the same planning, protocol, and verification stack. Although a scripted policy slightly outperforms our method on the easiest task, its success rate falls to 10% on the hardest, compared with 60% for our method.

Keywords: Language-guided manipulation · Safe robot execution · Collision-aware scene modeling · Model Context Protocol

1 Introduction

Robotic assistance in everyday environments requires robots to interpret ambiguous natural-language instructions and execute them safely and reliably in cluttered, contact-rich, and uncertain settings [6, 23, 24]. Consider the request in Fig. 1: *“Pour the water into the cup on the right.”* Executing this instruction requires (i) grounding linguistic references in physical entities and spatial

* Corresponding author.

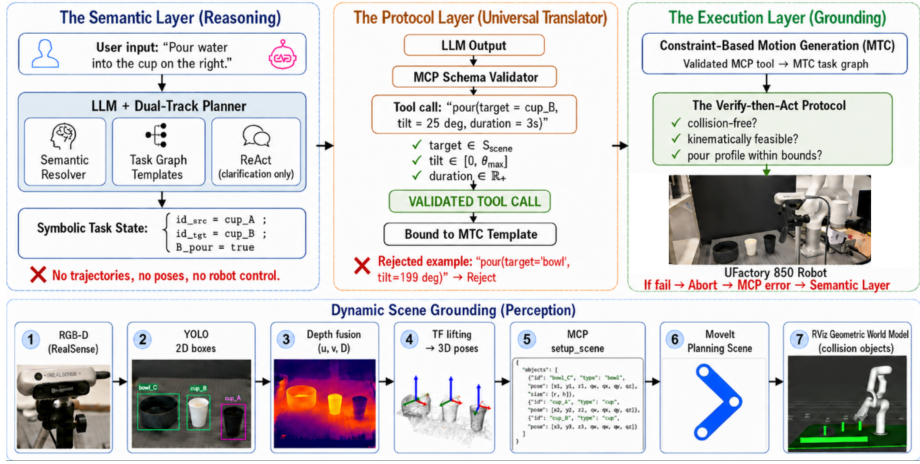


Fig. 1: Overview of the proposed framework. The Semantic Layer interprets the user instruction and produces a symbolic task state without generating poses or trajectories. The Protocol Layer validates MCP tool calls against a typed schema and binds valid calls to MTC templates. The Execution Layer follows a verify-then-act protocol, executing only motions that satisfy collision, kinematic, and task-specific constraints. The perception pipeline (bottom) grounds RGB-D observations into an explicit 3D MoveIt planning scene for collision-aware verification and execution.

relations, (ii) translating underspecified intent into a concrete, multi-step action sequence, and (iii) ensuring kinematic feasibility and collision avoidance during safety-critical manipulation tasks such as pouring and cooking [18, 30, 32]. The third requirement is particularly important for trustworthy robot execution. A plan may be linguistically correct yet physically unsafe, making it unsuitable for real-world deployment.

Recent work on language-guided manipulation has shown that LLMs can decompose natural-language instructions into symbolic plans or structured tool calls [1, 21, 25, 26]. However, a persistent language-action gap remains: a plan may be semantically plausible yet physically infeasible under kinematic and collision constraints [15]. One line of work pursues trustworthy robot execution through learned world models that predict the outcomes of candidate actions before execution. Although powerful, such models can hallucinate under partial observability and produce physically inconsistent predictions when they lack strong physics priors or explicit 3D structure, particularly in the safety-critical settings considered here [2]. We therefore take a complementary, explicit approach: rather than predicting action outcomes with a learned dynamics model, we verify each proposed action against a reconstructed, collision-aware representation of the current scene. We formalize the planning-execution boundary as a typed contract, mapping language-level decisions into schema-validated tool calls through the Model Context Protocol (MCP) [3].

Each validated call is then deterministically grounded in a MoveIt Task Constructor (MTC) [12] pipeline, which checks kinematic feasibility and collisions before any motion is executed on the robot.

Contributions. This paper makes three contributions:

1. **A typed planning contract that bridges the language-action gap.** Our core contribution is a validated interface between language reasoning and robot execution: language-level decisions are mapped to schema-validated tool calls, so malformed or unsupported commands are rejected before reaching the motion planner. We implement this interface through MCP as a portable schema layer. Because the interface exposes semantic object references and bounded skill-level parameters rather than poses or trajectories, it can extend across skills by adding a corresponding tool schema and motion template without modifying the planner or execution stack.
2. **Verify-then-act grounding with explicit pre-execution checks.** Each validated tool call is deterministically grounded in an MTC template, and every candidate motion is evaluated against an explicit, perception-grounded model of the current scene. Only motions that satisfy kinematic-feasibility and collision constraints are sent to the robot, providing an auditable safety gate between probabilistic language reasoning and physical execution.
3. **Evidence for structured interfaces and recovery.** We evaluate the system on real-robot pouring tasks involving liquids, granular media, discrete solids, and demonstrate transfer to a grasp-and-place task while retaining the same planning and verification stack. Comparisons with a scripted policy and an unstructured language-model baseline, together with ablations of safe-pose recovery and retry mechanisms, show that our proposed framework improves overall task execution reliability.

2 Related Work

2.1 Language-Guided Planning and Verified Execution

LLMs have increasingly been used as high-level robot planners that translate natural-language instructions into programs or structured action sequences. Code-as-Policies [21] composes predefined robot APIs into executable policies, while Plan-Seq-Learn [10] further combines language-level planning with motion planning and learned low-level control. Recent work couples language reasoning more directly with Task and Motion Planning (TAMP), which integrates symbolic task reasoning with geometric feasibility. AutoTAMP [8] translates language instructions into formal task representations for downstream TAMP and iteratively corrects invalid specifications. LLM³ [28] allows the LLM to propose symbolic actions and continuous parameters, using motion-planning failures for refinement, while LLM-GROP [33] incorporates visually grounded commonsense

reasoning into TAMP for long-horizon rearrangement. MoveIt Task Constructor (MTC) [12] provides a practical framework for constructing and validating multi-stage manipulation motions.

Our work differs primarily in how language reasoning is exposed to robot execution. The LLM is restricted to schema-validated tool calls containing semantic object references and bounded skill-level parameters; it does not directly specify robot poses, joint configurations, waypoints, or trajectories. Each accepted call is mapped to a predefined MTC template, with geometric motion generation and verification handled entirely by the execution layer. This design provides an explicit and auditable boundary between probabilistic language reasoning and geometric robot execution.

2.2 Perception and Scene Grounding

Language-guided manipulation requires linguistic references to be grounded in the observed environment. Prior work combines language with object detection, segmentation, pose estimation, or spatial representations to support visually grounded manipulation [13, 14, 27]. For collision-aware motion planning, these observations must additionally be represented in a consistent geometric frame.

In our system, RGB-D detections are transformed into the robot frame and inserted into the MoveIt planning scene as collision geometry. The LLM accesses this grounded scene through typed tools, providing a shared representation for semantic grounding and geometric verification while keeping perception and planning explicitly separated.

2.3 Robotic Pouring and Material Transfer

Robotic pouring remains challenging because liquid and granular dynamics are difficult to model and only partially observable. Prior work has explored reinforcement learning [4], model-based control [7], audio and haptic feedback [20], hierarchical imitation learning [31], and vision-based liquid perception [22, 34]. These methods primarily address perception or low-level control once the transfer objective is specified.

Our work addresses the complementary problem of executing language-specified transfer tasks through a structured and verified reasoning–execution interface. Rather than introducing a new pouring controller or dynamics model, we use pouring as a challenging physical domain for evaluating semantic grounding, structured action sequencing, pre-execution verification, and recovery.

3 Method

3.1 System Overview

We study language-guided pouring in domestic environments, where success requires both correct semantic grounding and physically feasible motion. An

instruction such as “pour the water into the cup on the right” demands resolving referential and spatial ambiguity, yet the resulting trajectory must also respect kinematic limits and collision constraints. End-to-end vision-language-action models aim to unify these capabilities, but balancing open-ended semantic reasoning with reliable physical execution remains challenging [5, 11, 24, 35].

We therefore make the reasoning-execution boundary explicit and typed: the language module produces structured, schema-validated action specifications, while the control stack executes only those that satisfy predefined feasibility and safety constraints. As illustrated in Fig. 2, the architecture comprises three layers. The Semantic Layer (Sec. 3.2) interprets user intent and outputs only semantic action requests, without specifying robot poses or trajectories. The Protocol Layer (Sec. 3.3) accepts only tool calls that satisfy the predefined schema and argument constraints. The Execution Layer (Sec. 3.4) maps each validated call to a predefined motion-planning template, generates the corresponding robot motion, and executes it only after kinematic and collision verification against the current planning scene. This separation keeps semantic reasoning independent while placing an explicit verification gate before physical execution.

3.2 The Semantic Layer

The Semantic Layer translates unstructured and potentially ambiguous instructions into a structured symbolic representation. Because unconstrained LLM outputs may contain invalid object references, unsupported actions, or out-of-range parameters, we organize this layer as a dual-track reasoning pipeline (Fig. 2).

Language-model instantiation. We use Claude Sonnet 4 as the reasoning backbone of the Semantic Layer. The model operates on natural-language instructions together with a compact, perception-grounded representation of the current scene rather than directly processing raw visual observations. RGB-D inputs are handled separately by the perception stack, which detects objects, estimates their geometry, and registers them in the planning scene. The resulting structured context, including grounded object identifiers, semantic classes, and relevant robot state, is provided to the language model for high-level reasoning and tool selection. Image interpretation, pose estimation, collision checking, and trajectory generation remain delegated to the corresponding perception and execution modules.

Semantic resolution and state abstraction. Taking pouring as an example, the semantic resolver maps a natural-language instruction U to a compact task-state tuple S for downstream planning:

$$S = \{\text{id}_{\text{src}}, \text{id}_{\text{tgt}}, B_{\text{pour}}\}, \quad (1)$$

where id_{src} denotes the source container, id_{tgt} denotes the target container, and both are symbolic object identifiers grounded in the perception-injected planning scene. B_{pour} is a Boolean flag indicating whether the instruction requires material transfer. For the instruction of Fig. 1, the resolver binds $\text{id}_{\text{src}} = \text{cup_A}$ and

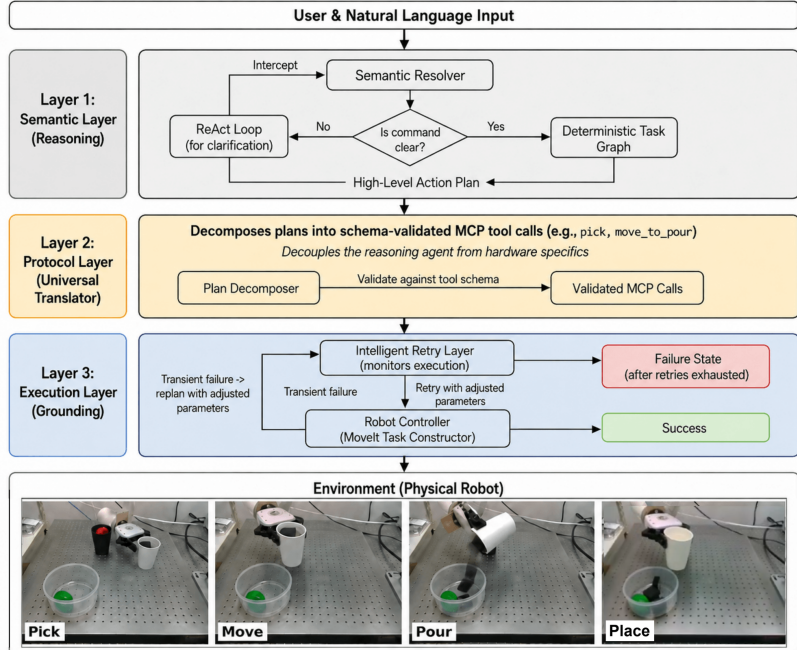


Fig. 2: Control flow through the three layers, complementing the contract view of Fig. 1. A semantic resolver routes each instruction either to a constrained ReAct loop that only queries information, or to a deterministic task graph. The protocol layer decomposes the resulting plan into schema-validated tool calls. The execution layer monitors execution and re-plans transient failures, committing to hardware only after verification. The bottom row shows the canonical pick, move, pour, and place sequence executing on the real robot.

$id_{tgt} = \text{cup_B}$, and sets $B_{pour} = \text{true}$. This abstraction retains only the task-relevant semantics, specifically which object transfers material to which target, while leaving poses, approach directions, and trajectories to the execution stack.

Dual-track policy. Based on instruction specificity, the planner follows one of two paths: a clarification track for ambiguous or underspecified requests and a deterministic execution track for well-specified commands. For ambiguous requests, the system invokes a constrained ReAct-style reasoning agent [29] that alternates between reasoning and information gathering. The agent may query perception or ask the user for clarification, but it cannot invoke actuation tools or issue motion commands. The clarification track is used specifically to resolve referential ambiguity rather than to perform long-horizon task planning. For example, given the instruction “pour the coffee into the cup” when two cups are visible, the agent queries the scene, identifies two matching object identifiers, and asks the user to specify the intended target. Only after clarification does it bind id_{tgt} and transfer control to the execution track. If the ambiguity remains unresolved, the task is halted rather than executed using a guessed target. For well-specified commands, such as the instruction in Fig. 1, the planner directly

selects a pre-validated finite-state-machine template that defines the canonical action sequence (pick, move, pour, place). The LLM determines the task-level intent but does not generate the execution sequence itself. Instead, execution follows a predefined motion template that is geometrically verified before being sent to the robot.

3.3 The Protocol Layer

Every action passes through a typed interface that enforces structural validity and maps each accepted call to a predefined motion-planning procedure. We implement this interface using the MCP, which exposes robot capabilities as a finite set of schema-defined tools. Each tool call is validated against its schema and scene-dependent preconditions before being passed to the execution layer [9, 19]. Tool calls contain semantic object references and bounded skill-level parameters, while robot poses, joint configurations, collision checking, and trajectory generation remain encapsulated within the corresponding motion-planning templates. This interface therefore provides an explicit barrier between probabilistic language reasoning and safety-critical robot execution.

Schema-based action definition. Each capability is exposed as a tool defined by a rigorous JSON schema. Let S_{scene} denote the set of object identifiers currently registered in the perception-grounded planning scene. A typical manipulation tool is defined over a typed domain,

$$\text{Tool}_{\text{pour}} : (\text{id}_{\text{tgt}} \in S_{\text{scene}}, \theta_{\text{tilt}} \in [0, \theta_{\text{max}}], \tau_{\text{duration}} \in \mathbb{R}^+). \quad (2)$$

At runtime, the MCP server applies three stages of validation. First, schema validation checks required fields and data types. Second, range validation ensures that physical parameters, such as tilt limits and action duration, remain within predefined admissible bounds. Third, referential validation verifies that semantic references, such as id_{tgt} , correspond to objects present in S_{scene} . Calls that fail any of these checks are rejected before reaching the motion planner, preventing invalid commands from propagating to the robot.

Atomic-to-composite binding. The Semantic Layer represents each tool call as an atomic action, while physical execution unfolds as a multi-stage motion program. Each tool is therefore associated with a predefined MTC template (Tab. 1). From the planner’s perspective, the action is atomic; from the executor’s perspective, it is decomposed into a sequence of motion stages. A validated call is passed to the corresponding task builder, which instantiates the template and searches for a kinematically feasible, collision-free solution. For example, a `pick` call expands into a sequence of approach, grasp, lift, and retreat stages that are verified before execution.

Auditability and platform agnosticism. The Protocol Layer also standardizes runtime behavior: every invocation is logged with its semantic parameters, and failures are normalized into structured responses such as `PlanningFailed` or `TargetUnreachable`, so the agent handles errors by retrying or querying the user, and never parses a driver exception. Because the interface hides the controller implementation, the same plan binds to different backends; we exploit this

Table 1: Mapping from MCP tools to MTC task graphs. Each atomic tool call at the planner level is bound to a composite pipeline of MTC stages at the executor level.

MCP tool interface (atomic)	Underlying MTC stage pipeline (composite)
<code>setup_scene</code>	Perception injection $\rightarrow$ collision-object spawning
<code>pick(object)</code>	Grasp generation $\rightarrow$ IK filtering $\rightarrow$ approach $\rightarrow$ close $\rightarrow$ lift
<code>move_to_pour</code>	Cartesian planning $\rightarrow$ tilt-constraint check $\rightarrow$ velocity scaling $\rightarrow$ execute
<code>place(target)</code>	Place-pose generation $\rightarrow$ approach $\rightarrow$ open $\rightarrow$ retreat

to bind tools first to a kinematic verification environment for pre-flight checking, then to the physical UFactory 850 driver.

3.4 The Execution Layer

The Execution Layer translates validated tool calls into safe, executable motions in three stages: it grounds the scene from perception, generates motion under explicit constraints, and verifies each candidate plan before committing it to hardware.

Dynamic scene grounding. Symbolic references such as `cup_A` must be grounded in geometry before the planner can reason about them. The experimental workspace shown in Fig. 3(a) comprises a UFactory 850 robot and an Intel RealSense D435i in an eye-to-hand configuration. As shown in Fig. 3(b), RGB streams are processed by a YOLO-based detector [16] under class-specific confidence thresholds, and detections are lifted into three dimensions by fusing aligned depth, median-filtered within each bounding box to suppress noise. The recovered points are transformed into the robot base frame via extrinsics from a 5×7 ChArUco board, accurate to sub-centimeter translation and one to two degrees. Rather than feeding these estimates to the controller, we inject them through the `setup_scene` tool, which instantiates collision geometries at unit scale using predefined cylindrical proxies rather than online-reconstructed meshes. The resulting perception-grounded planning scene (Fig. 3(c)) provides an explicit geometric representation of the current workspace for collision checking and kinematic reachability analysis.

Constraint-based motion generation. Rather than optimizing one trajectory end to end, MTC factorizes each skill into stages with explicit feasibility checks. *Generators* seed candidates: a grasp-pose generator reads the object pose from the planning scene and samples grasp hypotheses, pruned by inverse-kinematics (IK) reachability. *Propagators* solve local subproblems such as a safe-height approach, a descent, or a post-closure lift, each admitting its own velocity, acceleration, and orientation constraints. *Connectors* bridge disjoint states and backtrack to the next candidate when no feasible bridge exists, which keeps failure modes attributable to a stage.

For material transfer, we extend this pipeline with a conservative staged pouring profile [4, 7]. We do not model the detailed dynamics of the trans-

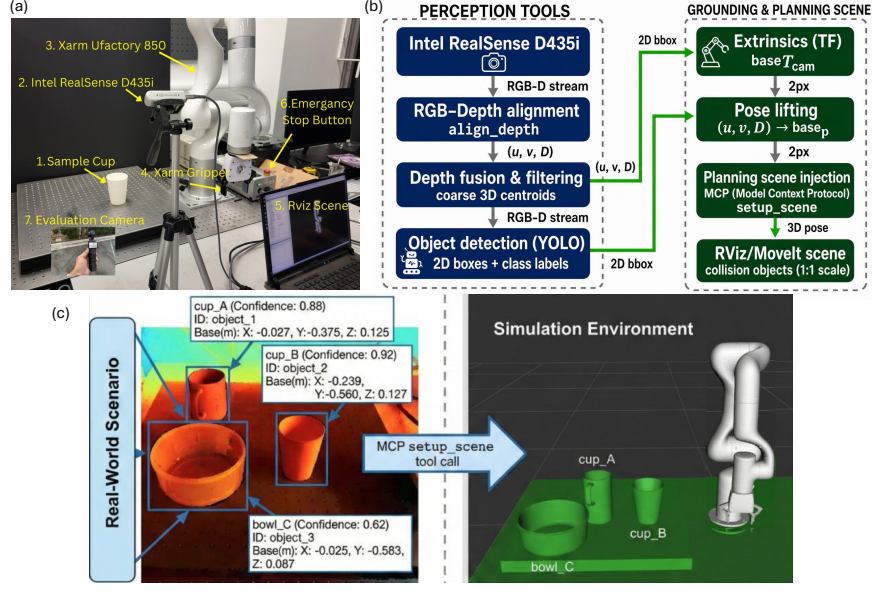


Fig. 3: (a) Real-world experimental setup using a UFactory 850 robot equipped with a two-finger gripper and an Intel RealSense D435i for perception. A separate camera records the experiments for evaluation. (b) Modular perception tools process RGB-D observations, transform detected objects into the robot base frame using calibrated extrinsics, and insert them into the planning scene through the `setup_scene` tool. (c) Detected objects and their estimated base-frame poses are shown on the colorized depth image (left), with the corresponding collision geometry in the planning scene (right).

ferred material. Instead, the end-effector executes a sequence of tilt-and-hold stages subject to predefined bounds on tilt angle, angular velocity, and dwell time, with each stage additionally checked for kinematic feasibility and collisions. These constraints limit abrupt motions during pouring while keeping the execution policy simple and inspectable. The verify-then-act mechanism therefore validates whether the commanded pouring motion is executable under the current kinematic and collision constraints, rather than predicting the resulting material dynamics.

Verify-then-act protocol. To reduce the risk of executing automatically generated plans in cluttered scenes, we enforce a strict kinematic and collision-verification step prior to hardware actuation [15, 17]. Once a candidate plan is produced, it is replayed against the reconstructed planning scene to verify that the full motion, from approach through retreat, remains continuous, reachable, and collision-free with respect to the modeled geometry. Only trajectories that pass these checks are serialized and transmitted to the UFactory 850 controller. If verification detects a collision, singularity, or discontinuity, execution is aborted and a structured failure is returned to the Semantic Layer. In addition, a predefined collision-checked safe pose is inserted between high-risk stages, for example after grasping and before pouring, to reduce error accumulation and

support recovery. This verify-then-act loop ensures that only motions validated for continuity, reachability, and collision avoidance in the reconstructed scene reach the hardware, preventing hallucinated or physically infeasible commands from being executed.

4 Experiments

4.1 Experimental Setup

All experiments are conducted on the UFactory 850 platform shown in Fig. 3(a), using the perception, protocol, and execution stack described in Sec. 3.

We evaluate four end-to-end pouring tasks of increasing difficulty (T1–T4), ranging from single-source pouring in a minimally cluttered scene to multi-source, multi-material, and cluttered scenarios. Each task is evaluated over ten independent physical trials. A trial comprises the complete pick, move-to-pour, pour, and place sequence and is counted as successful only if the material reaches the target container and the source container is returned. Because $N = 10$, each trial corresponds to ten percentage points; we therefore interpret small per-task differences cautiously and focus on consistent trends across tasks.

We compare our method with two baselines that share the same robot, perception pipeline, and motion-planning backend. The *FSM/BT* baseline executes a hand-crafted sequence of the same motion primitives, while the *LLM without task graph* retains the same language model, tool schemas, schema validation, safe-pose recovery, retry mechanism, motion-planning backend, and verify-then-act checks as our full method; only canonical task-graph sequencing is removed. End-to-end task success is the primary metric. Perception uncertainty is characterized separately through pose-grounding error in Fig. 4.

4.2 Perception and Scene Grounding

Grounding accuracy is evaluated by comparing estimated object poses with manually verified references across all scenarios. Position error is measured as the Euclidean distance between object centroids, and orientation error as the angular deviation between object frames (Fig. 4). Most estimates are sufficiently accurate for collision-aware planning and stable pouring, although larger errors occur under clutter and partial occlusion. Perception can introduce both semantic and geometric errors: visually similar containers may be associated with the wrong language reference, while pose-estimation errors can create discrepancies between the reconstructed scene and the physical workspace. The Protocol Layer rejects missing or inconsistent object references before motion planning, whereas the Execution Layer verifies candidate motions against the reconstructed planning scene. Residual pose uncertainty nevertheless remains a source of model–world mismatch, motivating explicit geometric verification and conservative execution.

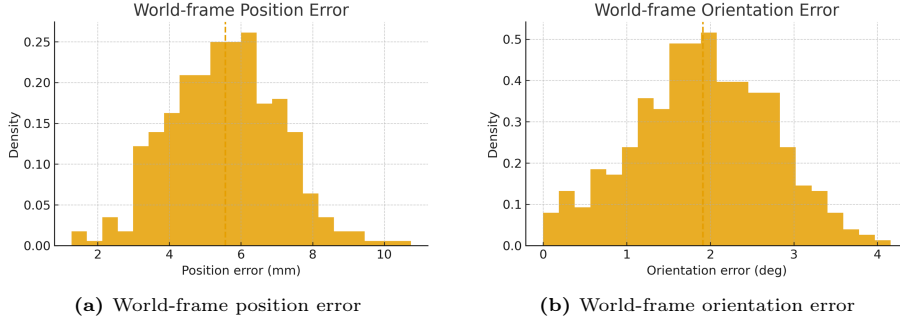


Fig. 4: World-frame object pose grounding errors across all scenarios. Both distributions exhibit non-negligible tails, motivating protocol-level verification prior to execution.

Table 2: Real-robot task performance. Each entry reports end-to-end success over 10 independent physical trials, with each trial corresponding to 10 percentage points. Per-task differences should therefore be interpreted cautiously. *FSM/BT* denotes a hand-crafted scripted policy, while *LLM w/o task graph* uses the same tool set without canonical action sequencing. Best result in each row is shown in bold.

Task	Ours	FSM/BT	LLM w/o task graph
T1: Single Pour	80%	90%	60%
T2: Multi-Source Pour	70%	40%	30%
T3: Multi-Material Pour	80%	60%	60%
T4: Complex Scene	60%	10%	40%
Mean	72.5%	50.0%	47.5%

4.3 Task Performance

We evaluate end-to-end performance on four real-robot tasks that progressively introduce longer horizons, denser clutter, and higher execution uncertainty (Fig. 5). Table 2 reports success against both baselines.

The ranking is not uniform, and the exception is informative. On T1, where the target is unambiguous and the scene is sparse, the scripted FSM/BT policy achieves 90% success compared with our 80%. In this simple setting, a hand-tuned sequence incurs no reasoning overhead, and the difference corresponds to a single trial. This advantage disappears on harder tasks: the scripted policy drops to 40% on T2 and 10% on T4, compared with 70% and 60% for our method, respectively. These tasks require adapting to distractors, rebinding references, and tighter geometric constraints that a fixed sequence cannot accommodate. The unstructured LLM exhibits the opposite weakness, performing below our method on every task because the absence of canonical sequencing allows grounding and ordering errors to propagate into invalid intermediate states. Averaged across all tasks, our method achieves 72.5% success, compared with 50.0% for the scripted policy and 47.5% for the unstructured LLM, with the largest gains occurring in the more constrained scenes.

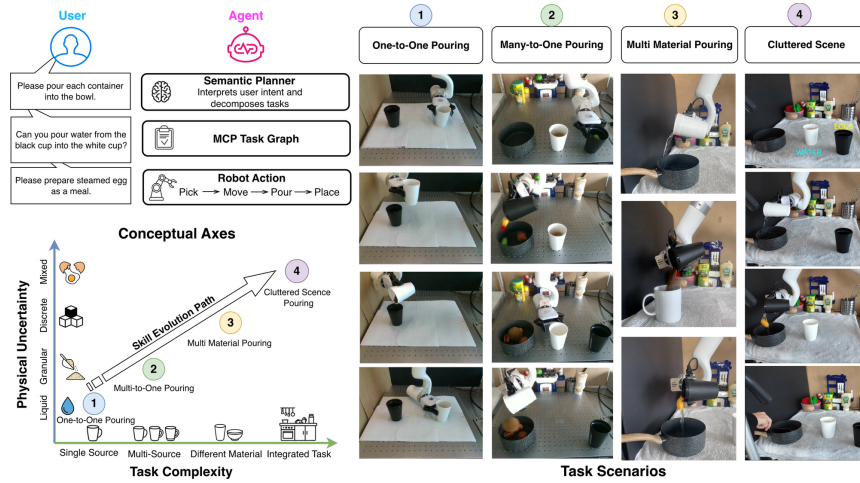


Fig. 5: Real-robot task suite. Left: two dimensions of increasing difficulty, task complexity (single source, multiple sources, different materials, and integrated task) and material uncertainty (liquid, granular, discrete, and mixed). Right: representative executions of the four evaluated scenarios, T1-T4.

4.4 Generalization Beyond Pouring

Since our contribution lies in the validated reasoning-execution interface rather than in pouring itself, we also test whether the same pipeline transfers to a different manipulation skill. We evaluate a fruit grasp-and-place task in which the robot moves an apple or orange into a bowl (Fig. 6). Transfer requires only a task-specific tool schema and corresponding MTC template; the planner, protocol layer, and verify-then-act mechanism remain unchanged. Since this task differs from the pouring suite in both objects and manipulation behavior, its results are not directly comparable to Tab. 2. Instead, the experiment tests whether the interface can be reused across skills.

Over ten trials, the system achieves 90% end-to-end success. The single failure was caused by a pose-estimation error near the camera field-of-view boundary, which resulted in a collision during grasp approach. Although this experiment is not intended as a broad multi-skill benchmark, it provides evidence that the framework can transfer to a new manipulation behavior by adding only a new tool schema and MTC template while retaining the same validation and verify-then-act mechanisms.

4.5 Multi-Material Robustness

T3 directly tests robustness to material variation by transferring water, coffee beans, and discrete solids such as eggs, which exhibit substantially different flow characteristics and failure modes. The method achieves 80% success on T3 (Tab. 2), matching its performance on the simpler single-source T1. This suggests that performance is relatively insensitive to material type, since pouring

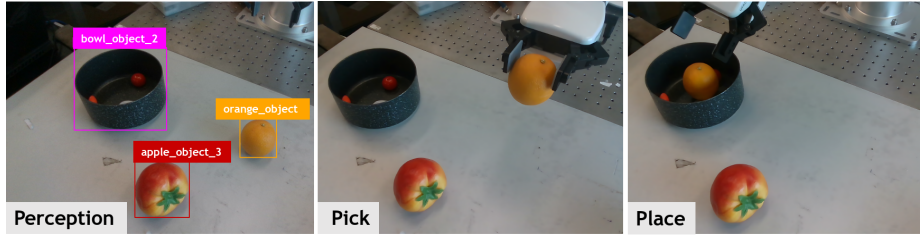


Fig. 6: Generalization to a fruit grasp-and-place task. Only the skill-specific tool schema and MTC template are changed. Perception grounds the fruit and bowl in the planning scene, while validated pick-and-place stages transfer the fruit to the bowl. The planner, protocol layer, and verification stack remain unchanged.

behavior is governed by parameterized tilt limits and execution-time verification rather than material-specific tuning. The remaining failures are also not strongly material dependent. Liquids benefit from conservative velocity scaling and stabilization at the pour pose, whereas granular and discrete materials are more tolerant of tilt timing but more sensitive to grasp stability; in these cases, failures are more often associated with grasping than with the pouring motion itself.

Across all three material classes, we use the same staged pouring profile and task-level interface without introducing material-specific planning logic. T3 achieves 80% end-to-end success despite the distinct transfer characteristics of liquids, granular media, and discrete solids. This result suggests that the same perception-planning-execution abstraction can generalize across substantially different material behaviors without task-specific retuning.

4.6 Ablation Studies

Table 2 compares alternative reasoning structures under shared perception and motion-planning backends; here, we further examine which protocol-level mechanisms contribute to end-to-end robustness. The scripted policy provides a lower bound on reasoning capability: without language-based adaptation, its success rate falls to 10% on T4, compared with 60% for our method. At the other extreme, the same language model without the task graph retains flexible reasoning but performs worse than our method on every task. Together, these comparisons suggest that language reasoning alone is insufficient; structured action sequencing and execution-time recovery are key contributors to reliable end-to-end performance.

Table 3 isolates the contribution of the two protocol-level recovery mechanisms by removing them individually while retaining schema validation and verify-then-act. Both have a larger effect on performance than the task graph: removing the safe-pose reset reduces average success by 32.5 percentage points, while disabling retry reduces it by 47.5 points, compared with a 25.0-point drop when the task graph is removed. Retry has the largest effect because many failures on this platform are transient. For example, an inverse-kinematics failure at

Table 3: Ablations of our own system, each over 10 independent physical trials. Removing the safe-pose reset or the retry mechanism leaves schema validation and verify-then-act intact, so these isolate the protocol-level recovery mechanisms.

Task	Ours	w/o safe-pose	w/o retry
T1: Single Pour	80%	50%	20%
T2: Multi-Source Pour	70%	20%	40%
T3: Multi-Material Pour	80%	70%	10%
T4: Complex Scene	60%	20%	30%
Mean	72.5%	40.0%	25.0%

a grasp pose or a planning failure caused by insufficient clearance can often be recovered by replanning with an alternative feasible candidate. Without retry, such failures terminate the trial. The safe-pose reset provides a complementary recovery mechanism by returning the arm to a predefined, collision-checked configuration between stages, reducing the likelihood of entering configurations from which subsequent motions cannot be planned.

These results indicate that robustness arises from both structured sequencing and protocol-level recovery, with recovery showing the larger effect in our experiments. Schema validation provides a front-end safeguard by rejecting malformed calls before they reach the motion planner, while recovery mechanisms address transient failures during physical execution. Canonical sequencing also contributes to overall reliability, although its measured effect is smaller. Given only ten trials per condition, individual per-task differences should be interpreted cautiously. We therefore limit our conclusions to the overall performance trends across tasks.

5 Conclusions and Future Work

We presented a typed reasoning-execution interface for language-guided manipulation that maps language-level intents into validated tool calls and predefined motion-planning templates. Rather than predicting physical outcomes with a learned dynamics model, the system verifies each candidate motion against a perception-grounded planning scene before execution, providing an explicit and auditable gate between probabilistic language reasoning and robot motion. Across four real-robot pouring tasks, our method achieves 72.5% mean end-to-end success, compared with 50.0% for the scripted FSM/BT baseline and 47.5% for the unstructured LLM baseline. Transfer to a grasp-and-place task reaches 90% success while retaining the same planning, protocol, and verification stack. Ablations further show that both structured sequencing and protocol-level recovery contribute to overall reliability, with retry producing the largest measured effect in our experiments.

The current approach remains limited by the accuracy of the reconstructed planning scene, does not model material dynamics, and lacks closed-loop feedback during execution. Future work will incorporate visual, tactile, and audio

feedback for closed-loop manipulation and material transfer, and evaluate robustness under greater occlusion, dynamic scene changes, and denser multi-object clutter.

References

1. Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., et al.: Do as I can, not as I say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691 (2022)
2. Ai, B., Tian, S., Shi, H., Wang, Y., Pfaff, T., Tan, C., Christensen, H.I., Su, H., Wu, J., Li, Y.: A review of learning-based dynamics models for robotic manipulation. *Science Robotics* **10**(106), eadt1497 (2025). <https://doi.org/10.1126/scirobotics.adt1497>
3. Anthropic: Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol> (2024), accessed 15 Aug 2026
4. Babaian, E., Sharma, T., Karimi, M., Sharifzadeh, S., Steinbach, E.: PourNet: Robust robotic pouring through curriculum and curiosity-based reinforcement learning. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 9332–9339 (2022)
5. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., et al.: RT-1: Robotics transformer for real-world control at scale. arXiv preprint arXiv:2212.06817 (2022)
6. Brohan, A., Chebotar, Y., Finn, C., Hausman, K., Herzog, A., Ho, D., Ibarz, J., Irpan, A., Jang, E., Julian, R., et al.: Do as i can, not as i say: Grounding language in robotic affordances. In: *Conference on robot learning*. pp. 287–318. Pmlr (2023)
7. Chen, T., Huang, Y., Sun, Y.: Accurate pouring using model predictive control enabled by recurrent neural network. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 7688–7694 (2019)
8. Chen, Y., Arkin, J., Dawson, C., Zhang, Y., Roy, N., Fan, C.: Autotamp: Autoregressive task and motion planning with llms as translators and checkers. In: *2024 IEEE International conference on robotics and automation (ICRA)*. pp. 6695–6702. IEEE (2024)
9. Crouse, M., Abdelaziz, I., Astudillo, R., Basu, K., Dan, S., Kumaravel, S., Fokoue, A., Kapanipathi, P., Roukos, S., Lastras, L.: Formally specifying the high-level behavior of LLM-based agents. arXiv preprint arXiv:2310.08535 (2023)
10. Dalal, M., Chiruvolu, T., Chaplot, D., Salakhutdinov, R.: Plan-seq-learn: Language model guided rl for solving long horizon robotics tasks. In: *International Conference on Learning Representations*. vol. 2024, pp. 11087–11115 (2024)
11. Driess, D., Xia, F., Sajjadi, M.S.M., Lynch, C., Chowdhery, A., Ichter, B., et al.: PaLM-E: An embodied multimodal language model. In: *International Conference on Machine Learning (ICML)* (2023)
12. Görner, M., Haschke, R., Ritter, H., Zhang, J.: MoveIt! Task Constructor for task-level motion planning. In: *IEEE International Conference on Robotics and Automation (ICRA)*. pp. 190–196 (2019). <https://doi.org/10.1109/ICRA.2019.8793898>
13. Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., Fei-Fei, L.: VoxPoser: Composible 3D value maps for robotic manipulation with language models. In: *Conference on Robot Learning (CoRL)* (2023)
14. Jiang, Y., Gupta, A., Zhang, Z., Wang, G., Dou, Y., Chen, Y., Fei-Fei, L., Anandkumar, A., Zhu, Y., Fan, L.: VIMA: Robot manipulation with multimodal prompts. In: *International Conference on Machine Learning (ICML)* (2023)

15. Jin, Y., Li, D., A, Y., Shi, J., Hao, P., Sun, F., Zhang, J., Fang, B.: RobotGPT: Robot manipulation learning from ChatGPT. arXiv preprint arXiv:2312.01421 (2023)
16. Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics> (2023), version 8.0.0; accessed 15 Aug 2026
17. Khan, A.A., Andrev, M., Murtaza, M.A., Aguilera, S., Zhang, R., Ding, J., Hutchinson, S., Anwar, A.: Safety aware task planning via large language models in robotics. arXiv preprint arXiv:2503.15707 (2025)
18. Kim, Y., Kim, D., Choi, J., Park, J., Oh, N., Park, D.: A survey on integration of large language models with intelligent robots. *Intelligent Service Robotics* **17**(5), 1091–1107 (2024)
19. Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., Li, Y.: API-Bank: A comprehensive benchmark for tool-augmented LLMs. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. pp. 3102–3116 (2023)
20. Liang, H., Li, S., Ma, X., Hendrich, N., Gerkmann, T., Sun, F., Zhang, J.: Making sense of audio vibration for liquid height estimation in robotic pouring. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 5333–5339 (2019)
21. Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., Zeng, A.: Code as policies: Language model programs for embodied control. In: *IEEE International Conference on Robotics and Automation (ICRA)*. pp. 9493–9500 (2023)
22. Lin, H., Fu, Y., Xue, X.: PourIt!: Weakly-supervised liquid perception from a single image for visual closed-loop robotic pouring. arXiv preprint arXiv:2307.11299 (2023)
23. Lin, Y., Comi, M., Church, A., Zhang, D., Lepora, N.F.: Attention for robot touch: Tactile saliency prediction for robust sim-to-real tactile control. In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 10806–10812. IEEE (2023)
24. Mon-Williams, R., Li, G., Long, R., Du, W., Lucas, C.G.: Embodied large language models enable robots to complete complex tasks in unpredictable environments. *Nature Machine Intelligence* **7**(4), 592–601 (2025)
25. Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., Garg, A.: ProgPrompt: Generating situated robot task plans using large language models. arXiv preprint arXiv:2209.11302 (2022)
26. Song, C.H., Wu, J., Washington, C., Sadler, B.M., Chao, W.L., Su, Y.: LLM-Planner: Few-shot grounded planning for embodied agents with large language models. arXiv preprint arXiv:2212.04088 (2023)
27. Wake, N., Kanehira, A., Sasabuchi, K., Takamatsu, J., Ikeuchi, K.: GPT-4V(ision) for robotics: Multimodal task planning from human demonstration. *IEEE Robotics and Automation Letters* **9**(11), 10567–10574 (2024)
28. Wang, S., Han, M., Jiao, Z., Zhang, Z., Wu, Y.N., Zhu, S.C., Liu, H.: Llm³: Large language model-based task and motion planning with motion failure reasoning. In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 12086–12092. IEEE (2024)
29. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2023)

30. Zhang, D.D., Zheng, Y., Fan, W., Lepora, N., Zhang, Z.: Self-supervised bayesian visual imitation learning applied to robotic pouring. In: 2024 IEEE International Conference on Industrial Technology (ICIT). pp. 1–7. IEEE (2024)
31. Zhang, D., Fan, W., Lloyd, J., Yang, C., Lepora, N.F.: One-shot domain-adaptive imitation learning via progressive learning applied to robotic pouring. *IEEE Transactions on Automation Science and Engineering* **21**(1), 541–554 (2022)
32. Zhang, D., Li, Q., Zheng, Y., Wei, L., Zhang, D., Zhang, Z.: Explainable hierarchical imitation learning for robotic drink pouring. *IEEE Transactions on Automation Science and Engineering* **19**(4), 3871–3887 (2021)
33. Zhang, X., Ding, Y., Hayamizu, Y., Altaweel, Z., Zhu, Y., Zhu, Y., Stone, P., Paxton, C., Zhang, S.: Llm-grop: Visually grounded robot task and motion planning with large language models. *The International Journal of Robotics Research* p. 02783649251378196 (2025)
34. Zhu, F., Hu, S., Leng, L., Bartsch, A., George, A., Farimani, A.B.: Pour me a drink: Robotic precision pouring carbonated beverages into transparent containers. *arXiv preprint arXiv:2309.08892* (2023)
35. Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., et al.: RT-2: Vision-language-action models transfer web knowledge to robotic control. In: *Conference on Robot Learning (CoRL)*. pp. 2165–2183 (2023)